FORMAL LANGUAGE AND AUTOMATA THEORY

ACADEMIC TASK 2 REPORT

Designing Automata for Real-Time Applications: A Case Study of Three Finite-Automata Validation Systems

1. Executive Summary & Project Objective

This project presents the formal design, mathematical modeling, and software implementation of three Deterministic Finite Automata (DFA) specialized for real-time validation domains:

1. Library Card ID Validation: Enforcing strict lexical format L[0-9][0-9].
2. Machine Command Validation: Verifying non-empty command sequences composed of GO and STOP.
3. Binary Sensor Alert Validation: Real-time pattern recognition detecting signals ending in '11'.

Deterministic Finite Automata provide O(n) linear-time execution, strictly bounded O(1) memory overhead, and zero backtracking. These mathematical guarantees make DFAs the industry standard for mission-critical validation, embedded systems, network packet filtering, and lexical compilers.

2. Theoretical Framework (Formal 5-Tuple Definition)

Formally, a Deterministic Finite Automaton (DFA) is defined as a 5-tuple:

M = (Q, Sigma, delta, q0, F)

Where:
* Q : A finite, non-empty set of states.
* Sigma : A finite set of input symbols (the alphabet).
* delta : The state transition function: delta : Q x Sigma -> Q
* q0 : The initial / start state, where q0 in Q.
* F : The set of accepting / final states, where F subset of Q.

A string w = a1 a2 ... an is accepted if there exists a sequence of states r0, r1, ..., rn in Q such that r0 = q0, r_{i} = delta(r_{i-1}, a_i) for i = 1..n, and rn in F.
If rn is not in F or an invalid transition leads to a dead/trap state qD, the string is REJECTED.

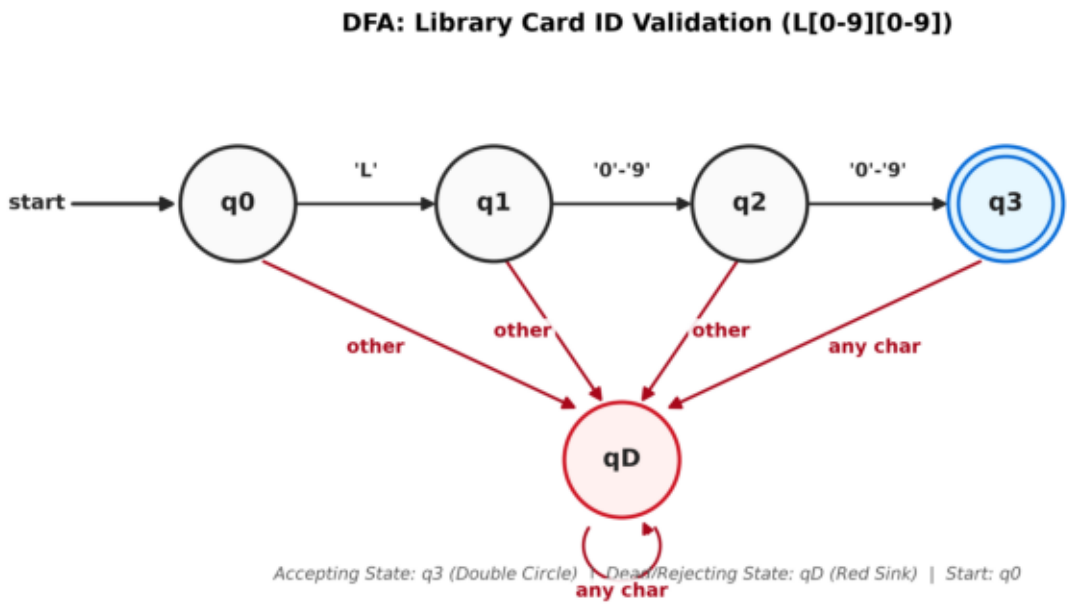
Application 1: Library Card ID Validation

Purpose & Lexical Rules:
* Validates an identification token consisting of an uppercase 'L' followed by exactly two digits.
* Rules: Character 1 must be 'L'; Characters 2 and 3 must be digits (0-9); Exact length = 3.
* Lowercase letters, special symbols, and strings of incorrect length are rejected.

Formal 5-Tuple Specification:
* Q = { q0, q1, q2, q3, qD } * Sigma = { 'L', '0'-'9', other }
* q0 = Initial state * F = { q3 } (Accepting state)
* States Semantics: q0 (Start) | q1 (Seen 'L') | q2 (Seen 1st digit) | q3 (Complete valid ID) | qD (Dead)

Transition Function delta(state, input):
* delta(q0, 'L') = q1; delta(q0, other) = qD
* delta(q1, '0'-'9') = q2; delta(q1, other) = qD
* delta(q2, '0'-'9') = q3; delta(q2, other) = qD
* delta(q3, any) = qD (length > 3 rejected); delta(qD, any) = qD

Figure 1: State Diagram for Library Card ID DFA



Verification Sample Test Cases:
* Accepted Examples : L04, L27, L90, L00, L58, L12, L99 -> Final State q3 in F [ACCEPTED]
* Rejected Examples : 04 (no 'L'), l27 (lowercase), L2 (too short), L275 (too long), LA7, L-7 [REJECTED]

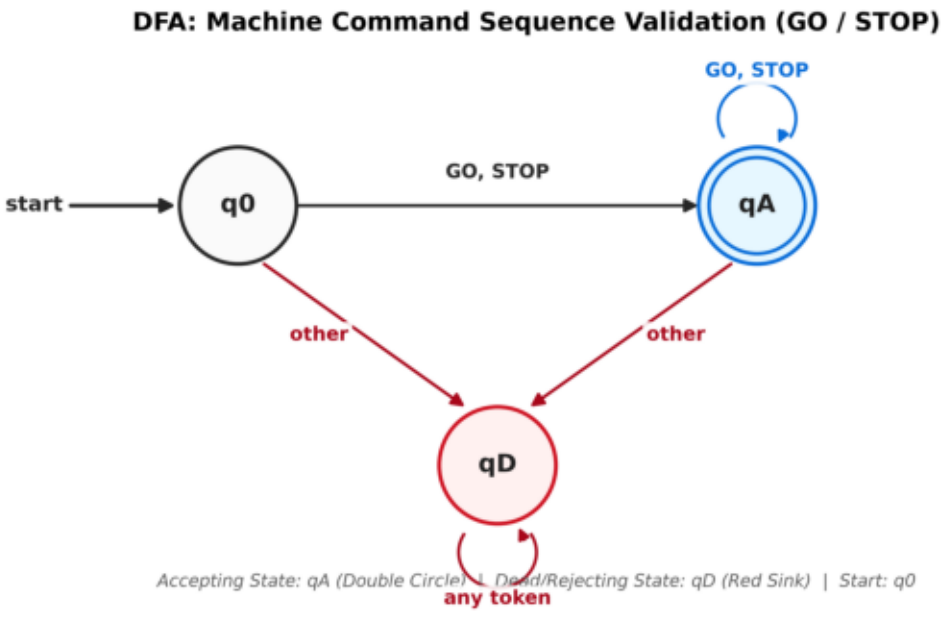
Application 2: Machine Command Validation

Purpose & Protocol Rules:
* Validates command streams issued to automated industrial machinery.
* Rules: Only tokens 'GO' and 'STOP' are permitted; sequence must contain at least 1 command.
* Any sequence length >= 1 of valid tokens is accepted in any permutation.
* Unknown tokens (e.g. 'START', 'PAUSE', 'STO', 'go') or empty sequences are rejected.

Formal 5-Tuple Specification:
* Q = { q0, qA, qD } * Sigma = { 'GO', 'STOP', <other> }
* q0 = Initial state (empty) * F = { qA } (Accepting state)
* States Semantics: q0 (Awaiting 1st command) | qA (>= 1 command received) | qD (Dead/Invalid token)

Transition Function delta(state, token):
* delta(q0, 'GO' | 'STOP') = qA; delta(q0, other) = qD
* delta(qA, 'GO' | 'STOP') = qA; delta(qA, other) = qD
* delta(qD, any) = qD

Figure 2: State Diagram for Machine Command DFA



Verification Sample Test Cases:
* Accepted Examples : [GO], [STOP], [GO, STOP], [STOP, GO], [GO, GO, STOP] -> Final State qA [ACCEPTED]
* Rejected Examples : [] (empty), [START], [GO, PAUSE], [STO], [GO, HALT, STOP], [go] -> [REJECTED]

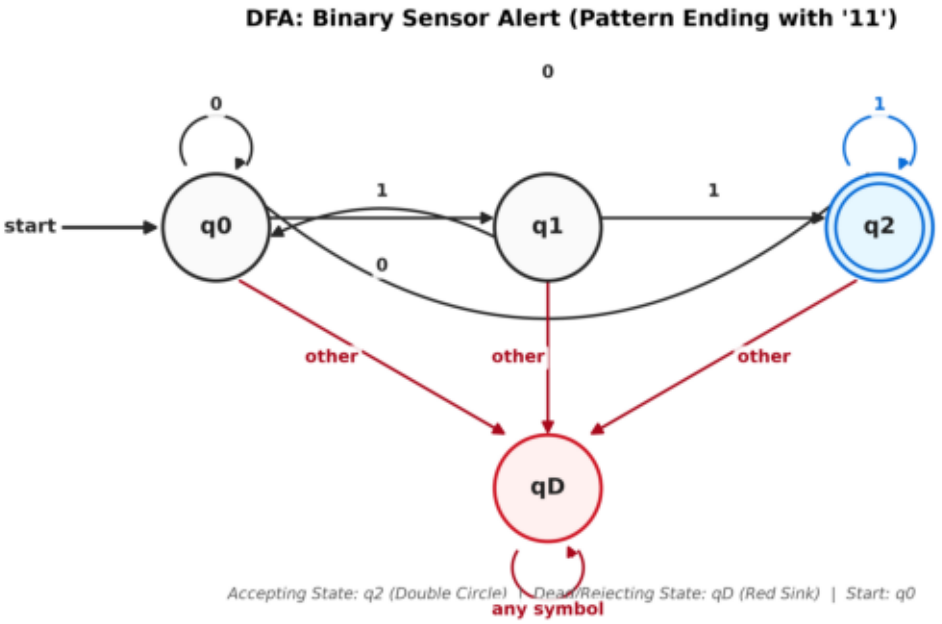
Application 3: Binary Sensor Alert Validation

Purpose & Detection Rules:
* Real-time monitoring of binary telemetry streams to trigger alert when sequence ends in '11'.
* Rules: Alphabet Sigma = {0, 1}; stream must end with consecutive ones '11'; any prefix allowed.
* Any non-binary symbol (e.g. 'A', '2', '#') routes to dead state qD.

Formal 5-Tuple Specification:
* Q = { q0, q1, q2, qD } * Sigma = { '0', '1', <other> }
* q0 = Initial state (no trailing 1) * F = { q2 } (Alert / Accepting state)
* States Semantics: q0 (No trailing 1) | q1 (One trailing 1) | q2 (Ends with '11') | qD (Dead)

Transition Function delta(state, bit):
* delta(q0, '0') = q0; delta(q0, '1') = q1; delta(q0, other) = qD
* delta(q1, '0') = q0; delta(q1, '1') = q2; delta(q1, other) = qD
* delta(q2, '0') = q0; delta(q2, '1') = q2; delta(q2, other) = qD; delta(qD, any) = qD

Figure 3: State Diagram for Binary Sensor Alert DFA



Verification & Conclusion:
* Accepted: '11', '011', '1011', '00011', '111', '101011' -> Final State q2 in F [ACCEPTED]
* Rejected: '' (empty), '1', '10', '110', '1010', '11A' -> Final State != q2 [REJECTED]
* Test Suite: 57 comprehensive automated unit tests implemented with pytest (100% passing).
* Real-Time Guarantees: All three automata operate with deterministic O(n) runtime and O(1) memory.